Strategy-Relative Regime Switching as a Verifiable RLVR Environment for Financial Planning

Imen

Draft (Preliminary) — April 2026

Abstract. We introduce the first verifiable reinforcement learning with verifiable rewards (RLVR) environment for financial planning, designed for LLM post-training. The environment uses the strategy-relative regime switching framework of Bilokon (2026), where an agent makes binary switching decisions under an observable mean-reverting signal with switching costs. We model signal dynamics with the Ornstein-Uhlenbeck (OU) process, exploiting its Gaussian transition density for exact Bellman solutions via Gauss-Hermite quadrature — bypassing the unknown-distribution obstacle of real markets. A parameter landscape analysis across 1,500 configurations identifies signal persistence (κ) as the dominant factor determining when planning diverges from myopic behavior (up to 22% of states). We evaluate Claude Opus 4 across 4 conditions (2 parameter zones $\times$ 2 prompt types, $N = 30$ each). The model achieves 97–99% accuracy on easy decisions. CoT-verified genuine planning occurs on 20% of hard decisions in the planning zone ($\kappa \in [0.1, 0.25]$) but only 4% in the control zone ($\kappa = 0.7$), consistent with planning being less valuable at fast reversion. A context prompt has negligible effect. The gym is open-source with single-function TRL integration.

1. Introduction

RLVR has transformed LLM post-training for math (DeepSeek-R1; DeepSeek-AI et al., 2025) and logic (Reasoning Gym; Stojanovski et al., 2025). But existing gyms test **static reasoning with complete information**. No gym tests **sequential decision-making under uncertainty** — where the agent acts before the full picture is revealed, each action has a cost, and the optimal strategy depends on future dynamics.

Financial planning faces a verification obstacle: real markets have unknown distributions, making optimal solutions uncomputable. We bypass this using the OU process, which provides simultaneously: (1) financially meaningful mean-reversion dynamics (Vasicek, 1977; Gatev et al., 2006); (2) Gaussian transitions enabling exact Bellman solutions; (3) bounded stationary distribution. The Gaussian property is what enables exact solutions — other mean-reverting processes (CIR, exponential OU, jump-diffusion) lack Gaussian transitions and would require approximate methods incompatible with strict RLVR verification.

We build on Bilokon (2026), who formalises the practitioner intuition that regimes are strategy-specific (not market-intrinsic) and proves that a single bit suffices as a predictive state variable for strategy performance.

Contributions. (1) First verifiable RLVR environment for sequential financial planning with exact Bellman solutions. (2) Parameter landscape characterisation across 1,500 configurations identifying when planning matters. (3) Two-bucket evaluation methodology decomposing comprehension from planning. (4) Four-condition evaluation showing 20% CoT-verified planning in the planning zone, 4% in the control zone, with negligible prompt effect.

---

2. Problem Formulation

The agent controls a binary regime $s_t \in \{0, 1\}$ adapted to filtration $(Z_0, \ldots, Z_t, s_{t-1})$, maximizing:

$$J(s) = \mathbb{E}\left[\sum_{t=0}^{T-1} u(s_t \cdot X_{t+1}) - \lambda \cdot \mathbf{1}\{s_t \neq s_{t-1}\}\right]$$

where $s_t = 1$ means the strategy is active, $s_t = 0$ means inactive, and λ is the switching cost. The signal follows an OU process: $Z_{t+1} = Z_t + \kappa(\theta - Z_t) + \sigma_z \epsilon_t$, with PnL $X_{t+1}|Z_t \sim \mathcal{N}(\alpha Z_t, \sigma_x^2)$. The Bellman recursion is solved exactly via backward induction with Gauss-Hermite quadrature.

Each episode samples different parameters $(\kappa, \lambda, \alpha, \sigma_z, T)$. The model knows α, λ, T but not κ or σ_z (must infer from observations). The Bellman solver knows all parameters — creating an information asymmetry discussed in Section 5.

---

3. Parameter Landscape and Planning Mechanism

Full factorial sweep: $5\kappa \times 5\lambda \times 5\alpha \times 3\sigma_z \times 4T = 1{,}500$ configurations, each solved exactly. The **solver-based disagreement metric** measures the fraction of states where the Bellman optimal differs from greedy, weighted by the OU stationary distribution.

Key findings. Signal persistence (κ) alone explains 46% of variance in disagreement ($R^2 = 0.456$). At $\kappa = 0.1$ (slow reversion), average disagreement is 13.1%; at $\kappa = 0.7$ (fast reversion), it drops to 1.2%. κ dominates over $\kappa \times T$ ($R^2 = 0.178$): the planning advantage comes from per-step persistence, not episode length. The λ/α ratio has a non-monotone effect, peaking at $[0.33, 0.50]$. Maximum disagreement is approximately 22% of states.

The planning mechanism. The greedy agent switches ON when $\alpha z > \lambda$ (threshold $z > \lambda/\alpha$). The optimal agent switches ON at a lower threshold because it amortises the switching cost over persistent future gains. At $\lambda/\alpha = 0.50$, the greedy threshold is $z > 0.50$; the optimal is $z > 0.22$. The **disagreement zone** between these thresholds is where planning changes the decision.

The sweep identifies the **planning zone**: $\kappa \in [0.1, 0.25]$ and $\lambda/\alpha \in [0.3, 0.5]$, where the gym produces meaningful training signal for planning.

---

4. Gym Architecture

Generator: Samples parameters, simulates OU trajectory, solves Bellman exactly. **Verifier:** Parses LLM decisions, computes regret-normalised score $\frac{J_{\text{LLM}} - J_{\text{random}}}{J_{\text{optimal}} - J_{\text{random}}}$. **Prompt Manager:** Multi-turn protocol — one signal per turn, model commits before seeing the next. **Integration:** Single function `reward_fn(completions, problems) → list[float]` compatible with TRL/veRL GRPO trainers.

---

5. LLM Evaluation

Setup. Claude Opus 4 evaluated across 4 conditions: 2 parameter zones (planning zone $\kappa \in [0.1, 0.25]$; control zone $\kappa = 0.7$) $\times$ 2 prompt types (original; context prompt adding "you are managing a mean-reversion strategy"). $N = 30$ episodes per condition, varied parameters per episode.

Results by condition:

Condition	κ	Prompt	Easy %	Hard decisions	Hard planning %
Planning zone	0.1–0.25	Original	97.1%	23	26.1% (6/23)
Planning zone	0.1–0.25	Context	97.7%	17	29.4% (5/17)
Control zone	0.7	Original	98.9%	12	25.0% (3/12)
Control zone	0.7	Context	98.0%	11	18.2% (2/11)

Easy accuracy is 97–99% across all conditions. The context prompt has negligible effect (26% vs 29% in planning zone).

CoT-verified planning (both prompts combined):

Zone	Hard decisions	Matched optimal	Genuine planning	Rate
Planning ($\kappa = 0.1$–0.25)	40	11	8	20.0%
Control ($\kappa = 0.7$)	23	5	1	4.3%
Total	**63**	**16**	**9**	**14.3%**

Every optimal match had planning reasoning in the CoT — zero lucky guesses. All 9 genuine planning instances show: trend recognition, horizon counting, amortisation logic, and greedy override.

Information asymmetry. The optimal policy knows κ; the model does not. The control zone ($\kappa = 0.7$) tests whether planning behavior responds to signal dynamics: genuine planning drops from 20% to 4%, consistent with the solver's disagreement rate ($\kappa = 0.7$: 1.2% vs $\kappa = 0.1$: 13.1%).

---

6. Discussion

The planning gap. Opus has near-perfect comprehension (97–99%) and plans genuinely on 20% of hard decisions in the planning zone. The capability exists but is not reliably activated — on 80% of hard decisions, the model defaults to myopic cost-benefit.

Context prompt. Qualitative strategy knowledge does not unlock planning. The gap is in constructing forward-looking reasoning from sequential observations, not in missing domain knowledge.

The 9 genuine instances. All show empirical reasoning: trend recognition, horizon counting, amortisation logic — the model plans like a trader, not a Bellman solver.

Implications for RLVR. RLVR training would provide gradient pushing the model to attend to signal history, estimate persistence, and override immediate cost-benefit when amortisation justifies it. The 9 instances prove this reasoning exists in the model's distribution.

Limitations. Single problem type (binary switching on scalar OU). Information asymmetry (solver knows κ, model does not). Small hard-decision counts per condition (11–23). No training results. Stationary process.

---

7. Conclusion

We presented the first verifiable RLVR environment for sequential financial planning. Claude Opus 4 genuinely plans on 20% of hard decisions in the planning zone, dropping to 4% in the control zone. The gap is not in knowledge (97–99% easy accuracy) but in reliably constructing forward-looking reasoning. The gym provides exact verification, a diagnostic methodology, and a concrete baseline. One function integrates it into any GRPO pipeline.

References

Bilokon, P. (2026). Strategy-relative market regimes as filtration compressions. SSRN 6504227.

DeepSeek-AI et al. (2025). DeepSeek-R1: Incentivizing reasoning capability in LLMs via RL. arXiv:2501.12948.

Gatev, E., Goetzmann, W., and Rouwenhorst, K. (2006). Pairs trading. *Rev. Fin. Studies*, 19(3):797–827.

Stojanovski, D. et al. (2025). Reasoning Gym. arXiv:2505.24760.

Vasicek, O. (1977). An equilibrium characterization of the term structure. *J. Fin. Econ.*, 5(2):177–188.

Appendix: Prompt Design

A.1 Original Prompt (Setup)

The setup prompt is sent once at the start of each episode. Parameters are substituted per episode.

```
You are managing a trading strategy over T={T} steps.

RULES:
- At each step t, you choose s_t = 1 (ON) or s_t = 0 (OFF).
- If ON (s_t=1): you earn the PnL for that step.
  Expected PnL = {alpha} x Z_t.
- If OFF (s_t=0): you earn nothing (PnL = 0).
- Every time you SWITCH (change from ON to OFF or
  OFF to ON), you pay a cost of {lambda}.
- If you stay in the same state, no switching cost.

YOUR GOAL: Maximize total profit = sum of PnL earned
minus switching costs paid.

PARAMETERS:
  Signal strength: alpha = {alpha}
  Switching cost:  lambda = {lambda}
  Starting state:  s_{-1} = {initial_regime} (ON/OFF)
  Horizon:         T = {T} steps

You will receive one signal observation Z_t at a time.
State your decision as: s_t = 0 or s_t = 1
```

A.2 Context Prompt The context prompt adds domain knowledge about mean-reversion. It replaces the first line and adds a second sentence:

```
You are managing a mean-reversion trading strategy over
T={T} steps. The signal Z_t represents a quantity (such
as a spread or deviation) that tends to revert toward
zero over time. You do not know how fast it reverts or
how noisy it is - you must estimate this from the
observations.
```

The remaining rules, parameters, and instructions are identical to the original prompt. As reported in Section 5, this additional context has negligible effect on planning rates (26% vs 29% in the planning zone).

A.3 Step Prompt (Per-Turn) At each time step, the model receives a single observation message:

```
t={t} | Z_t = {z_t} | You are currently ON/OFF
(s_prev = {prev_regime})
```

The model must commit a decision before seeing the next signal — enforcing the information constraint that matches the Bellman solution's filtration.

A.4 Scaffolded Prompt (Manual Test) The scaffolded prompt (tested on a single episode, seed=47) pre-computes decision options and highlights trends:

- Displays pre-computed **Option A** (stay) and **Option B** (switch) with exact expected PnL
- Highlights recent signal trend (e.g., "6 consecutive steps trending up")
- Adds instruction: "Consider whether the signal is likely to persist"
- Shows remaining steps in the horizon

Result: 25/25 correct including the hard decision. But the scaffold performed the planning for the model — it followed instructions rather than constructing the reasoning independently. This confirms the capability is latent but not a valid evaluation of planning.

A.5 Reward Function Integration with GRPO trainers uses a single function:

```
def reward_fn(completions_batch, problems_batch):
    verifier = RegimeSwitchingVerifier()
    return [
        verifier.score(completions, problem,
                       mode="trajectory")
        for completions, problem
        in zip(completions_batch, problems_batch)
    ]
```

The verifier parses LLM decisions from multi-turn text, computes realised payoff, and returns a regret-normalised score clipped to $[-1, 2]$.